JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Building RESTful Web Services

Table of Contents

UNIT 7: Building RESTful Web Services

Objectives

In this Unit you will learn –

- Understand the concept of RESTful web services and its principles, including the use of HTTP methods (GET, POST, PUT, DELETE) to perform operations on resources.
- Learn how to create RESTful APIs using Spring Boot by defining REST controllers and mapping HTTP requests to specific methods.
- Gain proficiency in handling different types of HTTP requests, such as GET, POST, PUT, and DELETE, using appropriate annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping`.
- Explore various techniques to extract data from HTTP requests, such as retrieving data from request parameters, path variables, and request bodies using annotations like `@RequestParam`, `@PathVariable`, and `@RequestBody`.
- Acquire the ability to design and implement RESTful services that provide CRUD (Create, Read, Update, Delete) operations on resources, leveraging the power of RESTful principles and Spring Boot's features.

Learning Outcome:

Learn about RESTful Web Services

7. Building RESTful Web Services

1. Introduction to RESTful Web Services

RESTful web services are a popular architectural style for designing and implementing web APIs (Application Programming Interfaces). REST, which stands for Representational State Transfer, provides a set of guidelines and constraints for building scalable, interoperable, and stateless web services. It is widely used in modern web development to create APIs that can be consumed by various clients, including web browsers, mobile applications, and other web services.

Key principles of RESTful web services:

1. Stateless: RESTful services are stateless, meaning that each request from a client to a server contains all the information necessary to understand and process that request. The server does not store any session or client-specific data between requests. This design principle allows for better scalability and fault tolerance.

2. Client-Server Architecture: RESTful web services follow a client-server model, where the client is responsible for initiating requests and the server is responsible for processing those requests and returning responses. This separation of concerns allows for independent development and scalability of both the client and server components.

3. Uniform Interface: RESTful services adhere to a uniform interface, which provides a set of standard HTTP methods (GET, POST, PUT, DELETE) to perform operations on resources. Each resource is identified by a unique URL (Uniform Resource Locator), and the HTTP methods are used to interact with these resources.

4. Resource-Oriented: RESTful web services are resource-oriented, meaning that they expose resources (such as data entities) that can be accessed and manipulated by clients. Resources are typically represented in a specific format, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language), and clients can perform operations on these resources using the standard HTTP methods.

Benefits of RESTful web services:

1. Scalability: RESTful services are inherently scalable due to their stateless nature. Multiple instances of the service can be deployed and load-balanced to handle a large number of client requests.
2. Interoperability: RESTful web services can be consumed by clients written in different programming languages and running on different platforms. They communicate over standard protocols like HTTP, making them highly interoperable.
3. Simplicity: RESTful services follow a simple and intuitive design approach, leveraging the existing capabilities of the HTTP protocol. This simplicity makes them easier to understand, develop, and maintain.
4. Caching: RESTful services take advantage of HTTP caching mechanisms, allowing clients to cache responses and reduce the load on the server. This improves performance and reduces network bandwidth consumption.
5. Wide Adoption: RESTful web services have gained widespread adoption and support in the industry. Many frameworks and tools provide built-in support for creating and consuming RESTful APIs, making it easier to develop robust and efficient web services.

RESTful web services provide a scalable, interoperable, and stateless approach to building web APIs. They leverage the HTTP protocol, adhere to a uniform interface, and focus on resource-oriented interactions. By following the principles of REST, developers can create efficient and flexible web services that can be easily consumed by a variety of clients.

2. Initializing RESTful Web Services

Spring Boot is a popular Java framework that simplifies the development of standalone, production-grade Spring applications. It provides a convenient way to initialize and configure RESTful web services with minimal boilerplate code. Here's a step-by-step guide to initializing RESTful web services using Spring Boot:

Step 1: Set up the Development Environment

Install Java Development Kit (JDK) on your machine.Set up your preferred Integrated Development Environment (IDE) for Java development, such as IntelliJ IDEA or Eclipse.

Step 2: Create a New Spring Boot Project

Use the Spring Initializr (https://start.spring.io/) to generate a new Spring Boot project. Select the required dependencies, including "Spring Web" for building RESTful web services. Alternatively, you can create a new Spring Boot project using your IDE's built-in project creation wizard.

Step 3: Project Structure

After generating the project, you'll have a basic project structure. The main source code will be located in the "src/main/java" directory.Create a package (e.g., "com.example.demo") to organize your application code.

Step 4: Create a REST Controller

Inside the package you created, create a new Java class and annotate it with @RestController. This annotation indicates that this class will handle RESTful requests and produce responses.

Define methods within the class to handle different HTTP requests. Annotate these methods with @RequestMapping to specify the URL mapping for each endpoint.

Step 5: Implement API Endpoints

Within each method in the controller class, write the logic to handle the respective HTTP request (GET, POST, PUT, DELETE). Use the appropriate annotations such as @GetMapping, @PostMapping, @PutMapping, or @DeleteMapping to define the HTTP method for each endpoint.

Step 6: Run the Application

In your IDE, run the Spring Boot application by executing the main method or using the provided run configuration. Spring Boot will automatically start an embedded web server (e.g., Apache Tomcat) and deploy your application.

Step 7: Test the Endpoints

Open a web browser or use API testing tools like Postman or cURL to send requests to your RESTful endpoints. Access the endpoints using the URL specified in the @RequestMapping annotations in your controller class.

Step 8: Customize and Enhance

Customize your RESTful web service by adding request parameters, path variables, request bodies, and handling different response types (JSON, XML). Configure additional Spring Boot features, such as database connectivity, security, or logging, as per your project requirements.

By following these steps, you can easily initialize RESTful web services using Spring Boot. Spring Boot simplifies the configuration and setup process, allowing you to focus on implementing the business logic of your APIs.

3. Creating RESTful Services

Rest Controller

A REST controller is a Java class that handles incoming HTTP requests and produces corresponding HTTP responses. It acts as the entry point for your RESTful services. To create a REST controller:

- Annotate the class with @RestController to indicate that it is a REST controller.
- Optionally, you can provide a base URL mapping using @RequestMapping

annotation at the class level to specify the common URL prefix for all endpoints in the controller.

Example:

```
@RestController
@RequestMapping("/api")
 public class UserController {
    // Controller methods will be defined here
}
```

Request Mapping

The @RequestMapping annotation is used to map HTTP requests to specific methods in a REST controller. It allows you to define the URL pattern for each endpoint and specify the HTTP method(s) that the method should handle.

- Use @RequestMapping annotation at the method level to specify the URL pattern for the endpoint.
- Specify the HTTP method using additional annotations such as @GetMapping, @PostMapping, @PutMapping, or @DeleteMapping.

Example:

```
@RestController
@RequestMapping("/api")
public class UserController {

    @GetMapping("/users")
    public List<User> getAllUsers() {
        // Logic to fetch and return all users
```

```
    }

    @PostMapping("/users")
    public User createUser(@RequestBody User user) {
        // Logic to create a new user
    }
}
```

Request Body

The @RequestBody annotation is used to bind the request body of an HTTP request to a method parameter in a REST controller. It is used when the client sends data in the body of the request, typically in JSON or XML format.

- Annotate the method parameter with @RequestBody to indicate that it should be populated with the request body.
- Spring automatically converts the request body into the appropriate Java object based on the content type of the request.

Example:

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    // Logic to create a new user using the data from the request body
}
```

Path Variable

The @PathVariable annotation is used to extract dynamic values from the URL path and bind them to method parameters in a REST controller. Path variables allow you to pass data in the URL itself.

- Specify the path variable name in curly braces {} within the URL pattern of the @RequestMapping annotation.
- Annotate the corresponding method parameter with @PathVariable and provide the same variable name.

Example:

```
@GetMapping("/users/{id}")
public User getUserById(@PathVariable("id") Long userId) {
    // Logic to fetch and return the user with the specified ID
}
```

Request Parameter

The @RequestParam annotation is used to retrieve query parameters from the URL of an HTTP request and bind them to method parameters in a REST controller. Query parameters allow clients to provide additional data in the URL.

- Annotate the method parameter with @RequestParam and specify the name of the query parameter.
- You can provide additional attributes like required, defaultValue, etc., to customize the behavior of the request parameter.

Example:

```
@GetMapping("/users")
public List<User> getUsersByRole(@RequestParam("role") String role) {
    // Logic to fetch and return users based on the specified role
}
```

By utilizing these annotations and concepts, you can create RESTful services with

Spring Boot that handle different types of requests, bind request data, and provide appropriate responses based on the client's needs.

4. REST API

REST (Representational State Transfer) APIs are a set of rules and conventions for building web services that follow the principles of REST. These APIs provide a standard way for clients to interact with server resources over HTTP. Here are the key HTTP methods used in REST APIs:

GET API

The GET method is used to retrieve data from the server. It is a safe and idempotent operation, meaning it should not have any side effects on the server and can be repeated multiple times without changing the server state. GET requests should only retrieve data and should not modify or create resources.

Example:

```
GET /api/users
```

This request retrieves a list of all users from the server.

POST API

The POST method is used to submit data to the server for creating new resources. It is not idempotent since submitting the same request multiple times will result in the creation of multiple resources with different identifiers. POST requests should include the data to be created in the request body.

Example:

```
POST /api/users

Request Body:

{
  "name": "John Doe",
  "email": "john@example.com"
}
```

This request creates a new user with the provided name and email on the server.

PUT API

The PUT method is used to update existing resources on the server. It is idempotent, meaning sending the same request multiple times will have the same effect as sending it once. PUT requests should include the full representation of the resource to be updated in the request body.

Example:

```
PUT /api/users/123

Request Body:

{
  "name": "Updated Name",
  "email": "updated@example.com"
}
```

This request updates the user with ID 123 on the server with the provided name and email.

DELETE API

The DELETE method is used to delete existing resources on the server. It is idempotent, meaning sending the same request multiple times will have the same effect as sending it once. DELETE requests should include the identifier of the

resource to be deleted in the URL.

Example:

```
DELETE /api/users/123
```

This request deletes the user with ID 123 from the server.

These HTTP methods form the foundation of RESTful APIs and allow clients to perform various operations on server resources. By implementing endpoints that handle these methods appropriately, you can build robust and scalable REST APIs.